FADENA
FACULTAD DE DEFENSA NACIONAL
Universidad de la Defensa Nacional

## Facultad de Defensa Nacional

## Licenciatura en Ciberdefensa

# Sistemas de tratamiento de Datos

## SQL Avanzado

**Año:** 2025

FADENA
FACULTAD DE DEFENSA NACIONAL
Universidad de la Defensa Nacional

# INTRODUCCIÓN

En clases previas vimos que SQL ofrece una serie de sentencias para interactuar con bases de datos, y que podían agruparse en diferentes categorías (DDL, DML, DCL). A continuación, veremos algunas posibilidades para estas que no se han desarrollado todavía.

# SENTENCIAS DE MANIPULACIÓN DE DATOS (DML)

Ya vimos que se pueden **hacer consultas a más de una tabla** mediante el uso de JOINS. Existen otras formas que pueden complementarse con estas, como las **subconsultas** y las **operaciones de conjunto**.

## SUBCONSULTAS

Una **subconsulta**, o **consulta anidada**, **es una sentencia SELECT dentro de otra sentencia SQL**, que puede ser otra **SELECT** o bien cualquier sentencia de manejo de datos: **INSERT**, **UPDATE**, **DELETE**.

Las subconsultas pueden anidarse unas dentro de otras tanto como sea necesario.

Tiene la misma sintaxis que una sentencia SELECT normal, exceptuando que **suele aparecer encerrada entre paréntesis**.

Podemos llamar **consulta externa o principal** a aquella que incluye a la subconsulta.

Generalmente es utilizada para simplificar consultas complejas al dividirlas en partes manejables, y realizar operaciones que de otro modo serían difíciles o imposibles.

Como cualquier SELECT, las subconsultas pueden retornar: **un único valor** (es decir una sola fila y columna), **una única fila** con una o más columnas, **múltiples filas** con múltiples columnas. Cada una de estas variantes se aplicarán según sea el caso.

Una subconsulta se puede encontrar en diferentes cláusulas:

- **Subconsultas en la cláusula WHERE:** Para filtrar registros basándose en los valores de la subconsulta.
- **Subconsultas en la cláusula HAVING:** Para filtrar grupos de datos según un criterio que depende de una subconsulta.
- **Subconsultas en la cláusula FROM:** Para utilizar el resultado de la subconsulta como una tabla temporal.
- **Subconsultas en la cláusula SELECT:** Para incluir un valor calculado por la subconsulta en el resultado.

## Subconsultas en las cláusulas WHERE y HAVING

Recordemos que el predicado que se evalúa para realizar una restricción en las cláusulas WHERE y HAVING está formado por comparaciones unidas por los operadores AND/OR. Cada comparación involucra dos **operandos** que pueden ser: **columnas, constantes (o expresiones que**

**contengan ambas)**, y ahora se agrega la posibilidad del **resultado de una subconsulta**.

Además de los dos operandos, cada operación se realiza con un **operador**. Hay una serie de operadores que se pueden utilizar con las subconsultas: **Operadores básicos de comparación** (=, <>, >, <, >=, <=), **IN, NOT IN, ANY, ALL, EXISTS y NOT EXISTS.**

- **Operadores Básicos de Comparación (=, <>, >, <, >=, <=):**

Los operadores básicos de comparación se pueden usar para comparar una expresión con el valor que devuelve una subconsulta.

**expresion operador ( subconsulta )**

En este caso **la subconsulta debe devolver un solo valor** (una fila con una columna). El predicado **será verdadero si la comparación indicada por el operador entre la expresión y el resultado de la subconsulta es verdadera**.

Por ejemplo, si queremos ver todos los productos que tienen un precio mayor o igual al producto más caro del fabricante “Samsung”.

```
SELECT *
  FROM producto
WHERE precio >= (SELECT MAX(precio)
                   FROM producto INNER JOIN fabricante
                     ON producto.codigo_fabricante  = fabricante.codigo
```

```
WHERE fabricante.nombre = 'Samsung');
```

En este caso, la subconsulta devolverá el mayor precio de los productos de Samsung, y se utilizará esta salida para la comparación con los precios de los productos de la consulta principal.

- **Operadores IN y NOT IN:**

El operador IN lo vimos anteriormente, especificando una lista de valores entre paréntesis. Otra forma de especificar esta lista de valores es con una subconsulta.

**expresion IN ( subconsulta )**

En este caso la subconsulta puede devolver varios valores, es decir **una columna y varias filas**, el predicado **será verdadero si la expresión es igual a alguno de los valores de la columna devuelta por la subconsulta**.

Por ejemplo, si queremos ver las ciudades en donde hay algún cliente:

```
SELECT ciudadID, nombre
  FROM ciudad
WHERE ciudadID IN ( SELECT ciudadID FROM clientes);
```

En este caso, la subconsulta devolverá la lista de identificadores de ciudad a la que pertenecen los clientes, y se usará para chequear si el identificador de las ciudades de la consulta externa está en esa lista.

También se puede **comprobar si un valor no está incluido en un conjunto de valores con NOT INT**.

**expresion NOT IN ( subconsulta )**

En este caso, el predicado **será verdadero cuando la expresión es distinta a todos los valores de la columna devuelta por la subconsulta**, o si esta no devuelve ninguna fila.

Por ejemplo, si queremos ver los clientes que no han realizado ningún pedido:

```
SELECT *
  FROM cliente
WHERE clienteID NOT IN (SELECT clienteID FROM pedido);
```

- **Operadores ALL y ANY.**

**ALL y ANY se utilizan con los operadores de comparación** (=, <>, >, <, >=, <=) y nos permiten **comparar una expresión con el conjunto de valores que devuelve una subconsulta**.

**expresion operador ANY ( subconsulta )**

En este caso, **la subconsulta debe devolver una columna y varias filas**. El operador es una comparación (=, <>, >, <, >=, <=). El predicado **será verdadero** si la comparación establecida por el operador es verdadera

**para alguno** de los valores de la columna devuelta por la subconsulta. En caso contrario se evalúa a falso.

Por ejemplo, si queremos ver los empleados con categoría "operario" cuyo sueldo sea mayor al de **algún** empleado con categoría "supervisor":

```
SELECT empleadoID, nombre
  FROM empleados
WHERE categoria = 'operario'
    AND sueldo > ANY (SELECT sueldo
                        FROM empleados
                      WHERE categoria = 'supervisor');
```

En este caso, la subconsulta devolverá la lista de sueldos de los supervisores, y se usará para chequear si el sueldo de cada operario es mayor a **alguno** de estos.

**expresión operador ALL ( subconsulta )**

En este caso, **la subconsulta debe devolver una columna y varias filas**. El operador es una comparación (=, <>, >, =, <=). El predicado se evalúa a verdadero si **la comparación** establecida por el operador **es verdadera para todos los valores de la columna devuelta por la subconsulta**. También se evalúa a **verdadero cuando la subconsulta no devuelve ninguna fila**. En caso contrario será falso.

Por ejemplo, si queremos ver los empleados con categoría "operario" cuyo sueldo sea mayor al de **todos los** empleados con categoría "supervisor":

```
SELECT empleadoID, nombre
FROM empleados
WHERE categoria = 'operario'
    AND sueldo > ALL (SELECT sueldo
                      FROM empleados
                      WHERE categoria = 'supervisor');
```

En este caso, la subconsulta devolverá la lista de sueldos de los supervisores, y se usará para chequear si el sueldo de cada operario es mayor a **todos** estos.

Hasta el momento vimos ejemplos en la cláusula WHERE, pero todos **estos operadores pueden ser usados en cláusulas HAVING**. Veamos por ejemplo el siguiente caso, donde se quiere obtener los departamentos y sueldo promedio, para los departamentos cuyo sueldo promedio sea mayor al promedio general:

```
SELECT departamentoID, AVG(sueldo)
  FROM empleados
GROUP BY departamentoID
HAVING AVG(sueldo) > ( SELECT AVG(sueldo)
                       FROM empleados );
```

En este caso, la subconsulta devuelve el sueldo promedio de todos los empleados, y se utiliza este valor para filtrar el resultado por grupo, mostrando aquellos departamentos que tengan sueldo promedio mayor.

**Sobre los operadores de subconsultas vistos hasta el momento**

**Cantidad de columnas devueltas por la subconsulta:** A fines prácticos para poder describir el uso de los operadores, se usaron siempre entre una única expresión y el resultado de una subconsulta que podía devolver una única columna y una o varias filas, según el caso.

Estos operadores se pueden usar también comparando un conjunto de valores, con la forma:

- (expr1, expr2, … ) operador ( subconsulta )
- (expr1, expr2, … ) [NOT] IN ( subconsulta )
- (expr1, expr2, … ) operador [ANY | ALL] ( subconsulta )

En estos casos, **la subconsulta debe devolver tantas columnas como las especificadas entre paréntesis a la izquierda**. Las comparaciones, se harán por cada campo en las filas de ambos lados.
Por ejemplo, si queremos ver las facturas con descuento y monto máximo (con operadores de comparación):

```
SELECT *
  FROM facturas
WHERE (descuento, monto) = ( SELECT MAX(descuento), MAX(monto)
                             FROM facturas );
```

Otro ejemplo, si queremos ver los clientes que compraron en algún mes en el que haya comprado un cliente en particular (con el uso de IN):

```
SELECT DISTINCT clienteID
  FROM facturas
WHERE (MONTH(fecha), YEAR(fecha)) IN ( SELECT MONTH(fecha), YEAR(fecha)
                                       FROM facturas
                                       WHERE clienteID = 282);
```

**Valores nulos con el uso de NOT IN y ALL:** En estos casos en particular, hay que tener en cuenta que, si la subconsulta devuelve algún nulo, el predicado se evaluará como nulo (más allá de que existan otras filas con valor), y la consulta externa no devolverá registros.

La solución para esto es incluir en la subconsulta una condición de "IS NOT NULL" para los campos a devolver, o bien usar funciones como COALESCE(columna, valor_si_es_nulo); que permite devolver un valor especificado para las filas donde columna sea nulo. Esta función devuelve el valor de la columna cuando no es nulo, y cuando es nulo devuelve el valor especificado a la derecha (puede ser otra columna o un valor explícito).

Entonces, por ejemplo, si queremos ver las facturas con descuento máximo, y sabemos que puede haber valores nulos en la columna "descuento", podríamos usar el operador ALL con:

```
SELECT *
FROM facturas
WHERE descuento >= ALL ( SELECT COALESCE(descuento, 0)
                         FROM facturas);
```

En este caso, la función COALESCE devolverá 0 para los valores de descuento que son NULL. Alternativamente se podría haber dejado en el SELECT el campo "descuento", e incluir una cláusula "WHERE descuento IS NOT NULL" para asegurar que no devuelva valores nulos.

- **Operadores EXISTS y NOT EXISTS**

El operador **EXISTS determina si una subconsulta devuelve o no alguna fila**. **La subconsulta** en este caso **hace referencia a columnas o expresiones de la consulta externa** para su ejecución.

**EXISTS ( subconsulta )**

En este caso, el predicado **será verdadero si la subconsulta devuelve al menos una fila**. Si no devuelve ninguna fila, será falso. En la ejecución de la subconsulta, en cuanto se devuelve la primera fila, se devuelve verdadero, sin terminar de obtener el resto de las filas.

Por ejemplo, si queremos ver los clientes que hayan hecho un pedido después del 10/08/2025, podemos hacer:

```
SELECT clienteID, nombre
  FROM cliente
WHERE EXISTS ( SELECT pedidoID
```

FADENA
FACULTAD DE DEFENSA NACIONAL
Universidad de la Defensa Nacional

```
                FROM pedido
            WHERE cliente.clienteID = pedido.clienteID
              AND pedido.fecha > '2025-08-10');
```

Ver que **la subconsulta hace referencia a un campo de la consulta externa (clienteID) para su ejecución**. Es decir, **para cada registro** (cliente) **de la consulta externa, se ejecuta la subconsulta** que verifica si para ese cliente existen pedidos después de la fecha indicada. Además, como **sólo importa si devuelve o no alguna fila (no los campos que pudiera devolver)**, puede escribirse una constante en el SELECT en lugar de * o cualquier columna (podría ser SELECT 1, por ejemplo).

**NOT EXISTS ( subconsulta )**

En este caso, el predicado será **verdadero si la subconsulta no devuelve ninguna fila**. Si devuelve al menos una fila, será falso. En la ejecución de la subconsulta, en cuanto se devuelve la primera fila, se devuelve falso, sin terminar de obtener el resto de las filas.

Por ejemplo, si queremos ver los clientes que no han realizado ningún pedido:

```
SELECT *
  FROM cliente
WHERE NOT EXISTS (SELECT id_pedido
                    FROM pedido
                   WHERE cliente.clienteID = pedido. clienteID);
```

En este caso se puede indicar lo mismo que para EXISTS. La subconsulta hace referencia a un campo de la consulta externa, y sólo importa si esta devuelve o no filas.

**Subconsultas correlacionadas y no correlacionadas**

Vimos recién que en el uso del operador EXISTS la subconsulta hace referencia a valores de la consulta externa. En los ejemplos previos, no se veían este tipo de referencias. En este sentido, podemos hacer una clasificación de las subconsultas:

- **Subconsulta no correlacionada: se ejecuta independientemente de la consulta externa**, recuperando datos sin depender de referencias a sus datos. En este caso, **la subconsulta se ejecuta primero (una sola vez)** y luego la consulta externa. Esto significa que el resultado de la subconsulta se utiliza como entrada para la consulta externa.

- **Subconsulta correlacionada: depende de los resultados de la consulta externa**, ya que **hace referencia a columnas o valores de esta**. En este caso, **la consulta externa se ejecuta primero** y, a continuación, **la consulta interna para cada fila devuelta por esta**. Cada vez que la consulta externa procesa una fila, la interna se ejecuta en relación con los valores de esa fila específica; de ahí el término de "correlacionada". Si bien la vimos recién con el uso de EXISTS, **las subconsultas correlacionadas pueden usarse con otros operadores**. Por ejemplo:

```
SELECT e1.nombre, e1.sueldo
    FROM empleados e1
WHERE sueldo > ( SELECT AVG(salario)
                 FROM empleados e2
                 WHERE e2.departamentoID = e1.departamentoID );
```

En este caso, la subconsulta hace una referencia a la tabla externa con "e1.departamentoID". El resultado final será la lista de nombre y sueldo de los empleados cuyo sueldo sea mayor al promedio de su departamento.

## Subconsultas en la cláusula FROM

Es posible incluir subconsultas en la cláusula **FROM**, **para realizar una consulta sobre la tabla que se obtiene como resultado de ejecutar otra consulta**. Son útiles para simplificar consultas de múltiples pasos. Cuando se una subconsulta en el FROM **se debe dar un alias a la tabla resultado de la subconsulta** mediante la cláusula AS. En este caso, **la subconsulta puede devolver tantas filas y columnas como sea necesario**.

El siguiente ejemplo encuentra el máximo precio promedio de productos agrupados por categoría:

```
SELECT MAX(promedio.precio_promedio)
  FROM ( SELECT categoria, AVG(precio) AS precio_promedio
         FROM producto
      GROUP BY categoria) AS promedio;
```

**La subconsulta resulta en una tabla (temporal o derivada)** renombrada como "promedio", con una fila por categoría de producto y el precio promedio. Esta se usa en el FROM de la consulta externa para obtener el máximo.

## Subconsultas en la cláusula SELECT

Las subconsultas pueden aparecer en la cláusula SELECT. En este caso, **tienen que ser subconsultas de tipo escalar, que devuelven una única fila y columna (un solo valor)**. Son útiles para mostrar información agregada relacionada a un registro devuelto.

Por ejemplo, supongamos que queremos ver el nombre de cada empleado y el salario promedio de su departamento:

```
SELECT nombre,
        ( SELECT AVG(salario)
            FROM empleados AS e2
          WHERE e2.departamento = e1.departamento ) AS salario_prom_dep
   FROM empleados e1;
```

La subconsulta calcula el sueldo promedio para cada departamento. Ver que "e2.departamento = e1.departamento" hace que la subconsulta sea correlacionada, es decir, se ejecuta por cada fila de la tabla principal empleados para encontrar el promedio del departamento de cada uno.

## Subconsultas en sentencias INSERT, UPDATE y DELETE

**El resultado de una subconsulta puede usarse como entrada de una sentencia INSERT**. En este caso, las columnas devueltas por la subconsulta deben coincidir con las con las especificadas en la sentencia INSERT INTO. Suele ser útil para crear copias de tablas, o para rellenar tablas con datos de otras basándose en criterios específicos.

Por ejemplo, si se tuviera una tabla para registrar por separado aquellos que aprobaron matemática, se podría cargar haciendo:

```
INSERT INTO aprobadosMatem (alumnoID, nombre, nota)
    SELECT a.alumnoID, a.nombre, c.nota
      FROM alumnos a INNER JOIN calificaciones c ON (a.alumnoID = c.alumnoID)
    WHERE c.nota > 6 AND c.materia = 'Matem';
```

En este caso en particular, la subconsulta puede no estar encerrada entre paréntesis.

Como las sentencias **UPDATE y DELETE** pueden usarse con la cláusula WHERE, estas **pueden utilizar las subconsultas** tal cual se mostraron en las sentencias SELECT.

Por ejemplo, si queremos aumentar un 10% el sueldo de los empleados de aquellos departamentos con un presupuesto mayor a 80.000:

```
UPDATE empleados
    SET sueldo = sueldo * 1.1
```

WHERE **departamentoID IN** ( SELECT departamentoID

FROM departamentos

WHERE presupuesto > 80000);

Si bien lo anterior se podría hacer con un JOIN entre las tablas, sirve como ejemplo.

De manera similar, podríamos querer borrar aquellos empleados que ganen menos que el promedio:

**DELETE** FROM empleados

WHERE **sueldo <** (SELECT AVG(sueldo) FROM empleados);

Ver tanto en todos los casos se usa el resultado de la subconsulta como entrada de la consulta principal.

## OPERADORES DE CONJUNTO

La **unión**, la **intersección** y la **diferencia** son operadores de álgebra relacional que se pueden usar en SQL. La sintaxis para esto es la siguiente:

sentencia_SELECT

**UNION | INTERSECT | EXCEPT [ ALL ]**

sentencia_SELECT

[ ORDER BY columna [ ASC | DESC ] [,columna [ ASC | DESC ] ];

Ver la cláusula ORDER BY sólo puede aparecer una vez en la consulta, al final de esta. La ordenación se realizará sobre el resultado de la unión, intersección o diferencia.

Para poder utilizar cualquiera de estos tres nuevos operadores, **las sentencias SELECT involucradas deben devolver el mismo número de columnas**, **y** las columnas correspondientes en ambas sentencias deberán ser **del mismo tipo de datos**.

- **Operador UNION**

Este operador devuelve como resultado todas las filas que devuelve la **primera sentencia SELECT, más aquellas filas de la segunda** sentencia SELECT que no han sido ya devueltas por la primera. En el resultado no se muestran duplicados. Se puede **evitar la eliminación de duplicados** especificando la **palabra clave ALL**.

La siguiente sentencia muestra las ciudades donde hay clientes o donde hay vendedores:

```
SELECT ciudad FROM clientes
UNION ALL
SELECT ciudad FROM vendedores;
```

En este caso, con el uso de ALL, devolverá las ciudades de ambas tablas sin eliminar duplicados (se mostrarán tantas veces como existan en ambas tablas).

- **Operador INTERSECT**

Este operador devuelve como resultado las filas que se encuentran tanto en el resultado de la **primera sentencia SELECT como en el de la segunda** sentencia SELECT. En el resultado no se muestran duplicados. Se puede **evitar la eliminación de duplicados** especificando la **palabra clave ALL**.

La siguiente sentencia muestra las ciudades donde hay clientes y también hay vendedores:

```
SELECT ciudad FROM clientes
INTERSECT
SELECT ciudad FROM vendedores;
```

- **Operador EXCEPT**

Este operador (que representa la operación diferencia o minus) devuelve como resultado las filas que se encuentran en el resultado de la **primera sentencia SELECT y no se encuentran en el resultado de la segunda** sentencia SELECT. En el resultado no se muestran duplicados. Se puede **evitar la eliminación de duplicados** especificando **la palabra clave ALL**.

Lo siguiente muestra ciudades donde hay clientes, pero no vendedores:

```
SELECT ciudad FROM clientes
EXCEPT
SELECT ciudad FROM vendedores;
```

La diferencia (EXCEPT) no es una operación conmutativa, mientras que el resto de los operadores de conjuntos sí lo son. Además, si bien los ejemplos mostrados son básicos, **las sentencias que participan en estas operaciones de conjunto pueden usar cláusulas WHERE, funciones, etc**.

### Sentencias equivalentes

Hay que tener en cuenta que, en muchas ocasiones, una misma consulta de datos puede responderse mediante distintas sentencias SELECT que utilizan operadores diferentes. Es decir, que puede llegarse al mismo resultado usando diferentes operadores con subconsultas, o bien operadores de conjunto, e incluso con el uso de JOINS entre tablas. Todas estas son formas de consultar a varias tablas y dependerá de las habilidades, conveniencias y necesidades. Además, todas estas se pueden combinar.

## SENTENCIAS DE DEFINICIÓN DE DATOS (DDL)

Habíamos visto que con las sentencias DDL se podían crear y eliminar esquemas, bases de datos, tablas, etc. También se pueden crear otros objetos de bases de datos no vistos hasta el momento.

### VISTAS

Una vista en SQL es una **tabla virtual basada en el resultado de una consulta SQL**. **No almacena datos físicamente**, sino que los datos existen

en una o más tablas referenciadas en la consulta. El contenido de una vista está definido como una consulta sobre una o varias tablas.

En SQL, la sentencia que permite definir una vista es la siguiente:

**CREATE VIEW** nombre_vista [ ( nombre_col, … ) ]

**AS** sentencia_SELECT;

Las columnas de la vista se pueden nombrar especificando la lista entre paréntesis. Si no se especifican nuevos nombres, los nombres son los mismos que los de las columnas de las tablas especificadas en la sentencia SELECT.

Las vistas son útiles por varias razones:

- **Seguridad:** permiten ocultar partes de la base de datos a ciertos usuarios. El usuario no sabrá que existen aquellos atributos (columnas) que se han omitido al definir una vista.
- **Reducción de complejidad y reutilización:** permiten simplificar consultas complejas sobre las tablas; y además guardarlas como vistas referenciándolas luego para su uso repetitivo, simplificando el código.
- **Independencia de datos a nivel lógico:** que se da cuando se reorganiza el nivel conceptual. Si se añade un atributo a una tabla, los usuarios no se percatan de su existencia si sus vistas no lo incluyen. Si una tabla existente se reorganiza o se divide en varias tablas, se pueden

crear vistas para que los usuarios la sigan viendo como al principio, aumentando la independencia de los programas respecto a los datos.

Por ejemplo, se puede definir una vista que muestre cada empleado con el nombre de su jefe:

```
CREATE VIEW v_emp_jef ( empID, empNombre, jefID, jefNombre )
    AS SELECT e.empID, e.nombre, j.empID, j.nombre
        FROM empleados e
         LEFT OUTER JOIN empleados j ON (e.jefeID = j.empID);
```

El usuario puede hacer uno de la vista posteriormente como si fuera una tabla normal. Por ejemplo:

```
SELECT v.ventasID, v.fecha, vej.empNombre, vej.jefNombre
  FROM ventas v
 INNER JOIN v_emp_jef vej ON (v.empleadoID = vej.empID)
WHERE ...;
```

Es importante recordar que las vistas se actualizan dinámicamente. Si los datos en las tablas subyacentes cambian, las vistas que se basan en esas tablas también cambiarán para reflejar los nuevos datos.

Como con otros objetos de la base de datos, su eliminación se lleva a cabo con la cláusula DROP:

```
DROP VIEW nombre_vista;
```

## ÍNDICES

Los índices son estructuras adicionales que **se utilizan para acelerar las operaciones de recuperación de datos de las tablas en respuesta a ciertas condiciones de búsqueda, basados en uno o más campos**. En lugar de escanear toda la tabla fila a fila, se puede utilizar los índices para encontrar las filas necesarias mucho más rápidamente. Es similar al índice de un libro: en lugar de leer todas las páginas, se puede ir directamente a la página que se necesita.

La sintaxis para la creación de índices es:

**CREATE INDEX** nombre_índice **ON** tabla (columna1, columna2, …);

Por ejemplo, si se quiere crear un índice para el campo apellido de la tabla empleados:

**CREATE INDEX** idx_apellido **ON** empleados (apellido);

Existen escenarios comunes en los que el uso de índices es importante para la recuperación de datos eficiente: búsqueda por campos particulares, ordenar o agrupar por campos, obtener subconjuntos de tablas grandes, unir tablas, etc.

Hay que tener en cuenta sin embargo **que crear índices excesivos puede tener el efecto contrario, ralentizando el rendimiento de la base de datos**. Cada índice requiere **espacio de almacenamiento adicional** y **aumenta el tiempo necesario para insertar, actualizar o eliminar datos (para el**

**mantenimiento de los índices)**. Es importante **encontrar un equilibrio** entre las ventajas de la indexación y sus posibles inconvenientes.

Se pueden tener en cuenta algunas ideas para la creación de índices:

- Indexar las columnas que se utilizan con frecuencia (por ejemplo, en cláusulas WHERE, JOIN y ORDER BY).
- Centrarse en las columnas donde los valores suelen ser distintos (alta cardinalidad). Si una columna tiene pocos valores posibles, un índice sobre esta no aportará ventajas de rendimiento significativas.
- No crear índices para tablas pequeñas (con pocos registros).

Según como se organicen, se pueden considerar dos tipos de índices:

**Índices agrupados (clustered):** Un índice agrupado **determina el orden físico de los datos de una tabla**. Cuando se crea un índice agrupado en una tabla, las filas se almacenan en el disco en el orden de la columna o columnas indexadas. **Como los datos sólo pueden ordenarse físicamente de una manera, sólo puede haber un índice agrupado por tabla**. Generalmente se crean para la clave primaria de la tabla. Algunos SGBD permiten el uso de la palabra clave CLUSTERED para su creación.

**Índices no agrupados (non clustered):** Los índices no agrupados **no afectan al orden físico de los datos**. En su lugar, **crean una estructura independiente que contiene punteros** a las filas de datos reales. **Una tabla puede tener varios índices no agrupados**, cada uno utilizado en un patrón de consulta distinto. Es el tipo de índice por defecto.

Existen además otro tipo de clasificación de índices:

**Índices únicos:** Un índice único garantiza que todos los valores de la columna o las columnas indexadas sean únicos, reforzando la integridad de los datos impidiendo valores duplicados (suele aplicarse a columnas de unicidad natural, por ejemplo, correo electrónico). Para esto utiliza la palabra clave UNIQUE:

**CREATE UNIQUE INDEX** idx_correo **ON** empleados (email);

**Índices compuestos:** Un índice compuesto es aquel que abarca más de una columna, por ejemplo:

**CREATE INDEX** idx_nombrecompleto **ON** empleados (apellido, nombre); creará un índice compuesto por las dos columnas.

Si se detectan índices redundantes, o que se solapan, o bien que ya no son necesarios, la sintaxis para eliminarlos dependerá del SGBD usado, pero en general es:

**ALTER TABLE** tabla **DROP INDEX** nombre_índice; o bien,

**DROP INDEX** nombre_índice **ON** tabla;

Esto eliminará el índice de la base de datos.

**OTROS OBJETOS DE BASES DE DATOS**

Además de los vistos hasta el momento, existen otros objetos que pueden crearse, modificarse y eliminarse en una base de datos mediante

CREATE, ALTER y DROP. **No profundizaremos en estos**, pero los mencionaremos para referenciar posibilidades.

**Procedimientos almacenados y funciones:** Ambos son bloques de código SQL pre compilados definidos por el usuario, que se almacenan en la base de datos y se ejecutan bajo demanda. Permiten reutilizar ese código en diferentes momentos.

**Procedimientos almacenados**: Pueden devolver valores (o no), aceptar parámetros de entrada y salida, y realizar operaciones de modificación de datos (INSERT, UPDATE, DELETE). Generalmente se usan para ejecutar lógica de negocio. No pueden usarse directamente en sentencias SQL directamente, son ejecutadas mediante comandos CALL o EXEC (EXECUTE). La sintaxis en general es:

```
CREATE PROCEDURE nombre_procedimiento
(param1 tipo_de_datos, param2 tipo_de_datos, ...)
AS
BEGIN
-- Sentencias SQL
END;
```

**Funciones**: Devuelven un valor y solo pueden tener parámetros de entrada. Se utilizan principalmente en sentencias SQL como parte de una expresión, por ejemplo, en la cláusula SELECT o WHERE. Ya vimos que los motores ofrecen una serie de funciones matemáticas, de fechas,

cadenas de caracteres, etc. SQL permite crear funciones definidas por el usuario (UDF), que pueden ser utilizadas de la misma manera. No puede actualizar datos, pero puede ser embebida directamente en consultas SQL. La sintaxis general es:

**CREATE FUNCTION** nombre_funcion

(**param1** tipo_de_datos, **param2** tipo_de_datos, …)

**RETURNS** tipo_de_datos

**AS**

**RETURN** <expresión>

**END**;

Nuevamente, se presentaron los procedimientos almacenados y las funciones **sólo a modo de ejemplo** de las diversas posibilidades que ofrecen los DBMS. Además, existen otros tipos de objetos que pueden crearse en la mayoría de los DBMS más importantes, como los **triggers** (disparadores), **asserts** (afirmaciones), etc., pero no serán tratados en este momento.

## SENTENCIAS DE CONTROL DE DATOS (DCL)

Habíamos visto que además de DDL y DML, había un grupo de control de datos denominado DCL.

El Lenguaje de Control de Datos o Data Control Language (DCL) en SQL es la parte dedicada a la seguridad, y es utilizado para controlar los

permisos y el acceso a la base de datos. DCL incluye dos comandos principales: **GRANT** y **REVOKE**, que se utilizan para asignar y quitar privilegios a usuarios o roles sobre objetos de la base de datos.

**GRANT**: Este comando **se utiliza para proporcionar privilegios a los usuarios**. Los privilegios determinan las acciones que un **usuario (o rol)** puede realizar en la base de datos. Estas acciones (privilegios) pueden incluir **SELECT, INSERT, UPDATE, DELETE, EXECUTE** y más. Los objetos a los que se otorgan los privilegios pueden ser **tablas**, **vistas**, **procedimientos almacenados** y más. La sintaxis básica es:

**GRANT** <privilegios> **ON** <objeto> **TO** <usuario>;

Por ejemplo:

GRANT SELECT, INSERT, DELETE ON empleados TO usuarioA;

le da a "usuarioA" el permiso de leer, insertar y eliminar datos en la tabla empleados.

**REVOKE:** Este comando hace exactamente lo contrario que GRANT. **Se utiliza para quitar los privilegios otorgados a un usuario**. La sintaxis es similar a la anterior:

**REVOKE** <privilegios> **ON** <objeto> **FROM** <usuario>;

Por lo general, sólo los administradores de base de datos o los usuarios con un alto nivel de privilegios tendrán la capacidad de ejecutar estos comandos.

## SENTENCIAS DE CONTROL DE TRANSACCIONES (TCL)

Si bien no este tipo de sentencias no figuraba en la clasificación vista hasta el momento, se comentará para mostrar sus posibilidades.

El Transaction Control Lenguage (**TCL**) se utiliza para **administrar las transacciones** en la base de datos, **permitiendo que una secuencia de operaciones SQL se ejecuten como una única unidad de trabajo**. Eso ayuda a asegurar la integridad de los datos.

**Una transacción SQL es una secuencia de una o más sentencias (INSERT, UPDATE o DELETE)** que se consideran una única unidad de trabajo única e indivisible (no tiene sentido que se ejecute una sin que se ejecuten las demás).

Los principios **ACID** (Atomicidad, Consistencia, Aislamiento y Durabilidad) son el conjunto de propiedades que garantizan que las transacciones en una base de datos sean procesadas de manera confiable.

- **A (Atomicidad):** Una transacción es una unidad de trabajo indivisible. O bien se completan todos sus pasos, o bien ninguno de ellos.
- **C (Consistencia):** Una transacción debe llevar la base de datos de un estado válido a otro. Si la base de datos estaba consistente antes de la transacción, debe permanecerlo después.
- **I (Aislamiento):** Las transacciones concurrentes no deben interferir entre sí.

- **D (Durabilidad):** Una vez que una transacción ha sido confirmada, sus cambios son permanentes y no se perderán, incluso en caso de un fallo del sistema.

Estos son **los principales comandos** del TCL:

**BEGIN TRANSACTION:** Marca el inicio de una transacción. Todas las operaciones posteriores formarán parte de esta transacción.

**COMMIT:** Este comando se utiliza para guardar de forma permanente todos los cambios realizados en la transacción. Una vez que se ejecuta un COMMIT, los cambios no se pueden deshacer con ROLLBACK.

**ROLLBACK:** Este comando se utiliza para deshacer todas las operaciones desde el último COMMIT o SAVEPOINT. Esto es útil para revertir la base de datos a un estado anterior en el caso de un error.

**SAVEPOINT:** Este comando se utiliza para identificar un punto en una transacción que puede ser usado para revertir parte de la transacción sin revertir todo. Es una forma de tener un ROLLBACK a un punto específico en la transacción en lugar de deshacer toda la transacción. Esto es útil en situaciones donde se tiene una transacción muy larga y queremos poder revertir parte de ella en caso de que algo salga mal, sin tener que deshacer toda la transacción. Cabe destacar que no todos los sistemas de gestión de bases de datos SQL soportan SAVEPOINT.

Sólo a modo de ejemplo, veamos una transacción con los comandos principales disponibles:

```
BEGIN TRANSACTION;  -- Inicia la transacción

-- Operación 1: Insertar datos de pedidos

INSERT INTO pedidos (id, cliente_id, total) VALUES (1, 101, 50.00);

-- Crear un punto de guardado antes de la siguiente operación crítica

SAVEPOINT punto_verificacion_1;

-- Operación 2: Actualizar el inventario

UPDATE inventario SET stock = stock - 1 WHERE producto_id = 101;

-- Si la operación 2 falla, se revierte solo hasta el punto de guardado

-- y no se pierden los datos insertados en la operación 1.

ROLLBACK TO SAVEPOINT punto_verificacion_1;

-- Operación 3: Se intenta una solución y se procede

COMMIT; -- Finaliza la transacción
```

## Otro ejemplo sin el uso de SAVEPOINT:

```
BEGIN TRANSACTION; -- Inicia la transacción

-- Lleva a cabo operaciones (en este caso actualiza 2 registros de cuentas)

UPDATE cuentas SET balance = balance - 100 WHERE cuentaID = 1;

UPDATE cuentas SET balance = balance + 100 WHERE cuentaID = 2;

COMMIT; -- Finaliza la transacción
```